ATELIER MAILLEUR

Utiliser, comprendre et développer Mailleur

Imports CAO et maillages, exports solveurs, API et publication

Version 1.0.0rc6

7 novembre 2026
9 chapitres et 4 annexes
Formats 2020+ : cible d'échange, périmètres documentés

Guide public destiné aux utilisateurs et contributeurs. Les formats intégrés, les passerelles conditionnelles et les validations externes sont distingués explicitement.

Sommaire

1. Ce que signifie la compatibilité 2020+ . . . 3

2. Installer et vérifier le logiciel . . . 4

3. Importer une pièce ou un maillage . . . 5

4. Générer, contrôler et exporter . . . 8

5. Utiliser l'API locale . . . 9

6. Architecture du parcours d'import . . . 10

7. Entraîner et partager un guide . . . 11

8. Développer et optimiser avec des preuves . . . 12

9. Distribuer et publier . . . 13

Annexe A — Formats et exports solveurs . . . 14

Annexe B — Imports natifs et passerelles . . . 19

Annexe C — Publication et contribution . . . 23

Annexe D — Validation de la rc6 . . . 26

Les liens et signets rejoignent les chapitres. Les annexes regroupent les formats, les passerelles natives, la publication et les preuves de validation de cette révision. Les fichiers Markdown restent les sources éditables de référence.

Ce guide décrit les interfaces et l'intégration de la révision rc6 du **7 novembre 2026**. Il complète le guide général, la liste des formats et le guide de publication. Les mesures Apple M3 du 4 novembre dans MESURES_M3.md concernent la révision mesurée à cette date : elles ne mesurent pas automatiquement les nouveaux lecteurs et exports.

Mailleur est une application locale de préparation géométrique et de maillage. Python fournit l'interface, les commandes, l'API HTTP et la lecture/écriture des formats. Rust assure les traitements numériques et pilote Gmsh/OpenCASCADE pour construire la géométrie et les maillages. Un petit réseau peut conseiller des recettes lorsque sa validation le permet ; le moteur classique reste responsable des nœuds, éléments et contrôles.

1. Ce que signifie la compatibilité 2020+

La cible du projet est l'échange avec les logiciels de CAO et de calcul utilisés depuis leurs versions 2020, sans priorité imposée à un seul éditeur. **Ce n'est pas une certification de tous les logiciels, formats et versions depuis 2020.** Il faut distinguer trois états :

État	Signification
Lecteur ou écrivain intégré	Le code peut traiter le format ou un sous-ensemble documenté, avec des tests sur des fichiers déterminés
Conversion locale nécessaire	Un logiciel ou traducteur externe doit être installé et configuré ; reconnaître l'extension ne suffit pas
Validation externe restant à faire	Le fichier a été contrôlé par Mailleur ou relu par un lecteur, mais pas encore exécuté dans le logiciel cible

Une version 2020 de SOLIDWORKS n'est pas supposée traduire complètement les fichiers natifs de toutes les versions plus récentes. Les extensions `.prt`, `.asm`, `.inp` ou `.dat` sont également ambiguës entre logiciels : le lecteur choisi et le contenu doivent correspondre, pas seulement le suffixe.

Un export de maillage n'est pas un modèle mécanique complet. Les matériaux, lois de comportement, épaisseurs, contacts, conditions aux limites, chargements et commandes de calcul restent à définir dans le solveur.

2. Installer et vérifier le logiciel

Prévoir Python 3.11, 3.12 ou 3.13. Si l'archive ne contient pas un binaire du noyau correspondant au système et à son architecture, installer aussi Rust/Cargo. Extraire le dossier complet dans un emplacement accessible en écriture, puis lancer depuis sa racine :

```
python scripts/install.py
python scripts/launch.py
```

Ici, `python` doit désigner une version prise en charge. Sur macOS, il peut être nécessaire d'employer explicitement `python3.12`. Les lanceurs `Lancer.command`, `Lancer_windows.bat` et `Lancer_linux.sh` suivent le même principe. Internet sert à installer les dépendances initiales ; les calculs fonctionnent ensuite localement.

Pour les commandes suivantes, utiliser l'interpréteur de l'environnement créé :

Système	Interpréteur depuis la racine du projet
macOS/Linux	`.venv/bin/python`
Windows	`.venv\Scripts\python.exe`

Exemples macOS/Linux :

```
.venv/bin/python -m atelier doctor
.venv/bin/python -m atelier formats --json
```

La première commande vérifie l'installation ; la seconde expose le catalogue d'import/export de la version exécutée. Ce catalogue est plus fiable qu'une liste copiée depuis un ancien document. Pour une extension conditionnelle, lire `conditional`, `available`, `converter` et `reason`. Un exécutable trouvé ne garantit pas à lui seul que sa licence, sa classe COM ou la version du document sont utilisables.

Les petits réseaux actuels s'entraînent sur CPU, en double précision. Leur taille ne nécessite pas un GPU dédié. La mémoire et le temps de la génération des maillages doivent toutefois être mesurés sur les propres pièces de l'utilisateur ; une démonstration de quelques milliers d'éléments ne dimensionne pas un assemblage industriel.

3. Importer une pièce ou un maillage

CAO et maillages intégrés

Le projet d'import contient les réglages géométriques et de maillage ; `--source` fournit le fichier à préparer. Par exemple, après adaptation des unités et du mode dans `examples/atelier_import.json` :

```
.venv/bin/python -m atelier mesh examples/atelier_import.json \
  --source /chemin/piece.step -o output/piece --formats msh,inp
```

Pour un fichier provenant d'un autre lecteur, remplacer la source tout en choisissant un mode compatible avec sa géométrie et ses unités. Le suffixe n'indique pas à lui seul s'il s'agit d'une peau ouverte, d'une surface fermée ou d'un domaine volumique maillable.

Famille d'entrée	Extensions du registre	Traitement
CAO d'échange	`.step`, `.stp`, `.iges`, `.igs`, `.brep`	Import de géométrie via Gmsh/OpenCASCADE
Peau STL	`.stl`	Parcours Gmsh de reconstruction surfacique
Maillage UNV	`.unv`	Lecture par Gmsh, puis préparation de peau dans Rust avant reconstruction
Surfaces polygonales	`.obj`, `.ply`, `.off`	Décodage du fichier, puis préparation numérique en Rust
Maillages généraux	`.msh`, `.vtk`, `.vtu`, `.med`, `.mesh`, `.meshb`, `.vol`	Lecture selon le format déclaré, puis reconstruction à partir de la peau
Maillages de solveurs	`.inp` pour le lecteur Abaqus ; `.bdf`, `.nas`, `.fem` pour le lecteur Nastran	Lecture du sous-ensemble de maillage reconnu, sans transfert automatique du modèle physique
Conteneur 3MF	`.3mf`	Lecture de la géométrie 3MF Core et des instances prises en charge

L'import d'un maillage a pour objectif de **produire une géométrie de reconstruction à remailler**. Il ne conserve pas automatiquement les identifiants des anciens éléments ni leurs groupes. Les noms et champs repérés par le lecteur sont consignés dans les métadonnées ; ils ne deviennent pas des chargements, des matériaux ou des groupes du nouveau maillage. Garder l'original pour comparer et recréer les sélections physiques nécessaires.

Les éléments, ordres et connectivités acceptés pour reconstruire la peau sont documentés dans FORMATS.md. Un lecteur de fichier peut décoder une cellule que le noyau refuse ensuite : le logiciel doit signaler ce refus plutôt que supprimer silencieusement la cellule. Les fichiers de solveurs avec `INCLUDE` externes doivent être rendus autonomes avant l'import.

Le préparateur Rust accepte les familles documentées de triangles, quadrangles, tétraèdres, hexaèdres, prismes et pyramides, avec plusieurs connectivités d'ordre supérieur. Pour ces dernières, il reconstruit une peau à partir des **sommets principaux** : c'est une approximation géométrique déclarée, pas un transfert éléments finis conservant exactement les nœuds intermédiaires et la courbure. Les quadrangles de peau sont triangulés. Les Jacobiennes courbes du maillage source ne sont donc pas certifiées par cette opération.

Les nouveaux tests de préparation couvrent des cellules linéaires tétraédriques, hexaédriques, prismatiques et pyramidales, ainsi que des cas `tetra10`, `triangle6` et `quad8`. Les autres variantes d'ordre supérieur reconnues par le contrat ne disposent pas chacune d'un essai individuel annoncé : ne pas transformer l'existence du mapping en validation de tous les maillages provenant de tous les solveurs.

En présence de cellules volumiques, les faces internes partagées sont retirées selon leur connectivité ; seules les faces de frontière servent à la reconstruction. Les anciennes interfaces internes, matériaux et ensembles ne sont pas recréés automatiquement. Le contrôle rejette notamment les indices invalides, les coordonnées non finies, les cellules

primaires inversées ou non convexes et les peaux non conformes aux contrôles topologiques. Il ne réalise pas une recherche exhaustive de toutes les auto-intersections possibles.

Particularités de 3MF Core

Le lecteur prend en compte les objets maillés, les composants internes au modèle, les éléments du `build`, leurs instances et leurs transformations. Il lit l'unité déclarée dans le modèle — millimètre par défaut en 3MF — et transmet au noyau la conversion vers l'unité d'entrée choisie dans le projet. Le noyau reste responsable de l'application numérique des transformations et du contrôle des coordonnées.

Le périmètre est géométrique : couleurs, textures, matériaux d'impression et réglages de fabrication ne deviennent pas des données de calcul mécanique. Les extensions obligatoires non gérées, les composants externes ou multifichiers, les objets de type autre que `model`, les cycles et les références absentes sont refusés. Le test entre instances repose sur leurs boîtes englobantes : des boîtes qui se touchent ou se chevauchent provoquent un refus conservateur, même si les formes réelles pourraient être séparées. Il ne s'agit ni d'un test exact d'intersection ni d'une union booléenne des objets.

Les limites courantes du lecteur incluent 200 Mio pour la source et le contenu décompressé, 4 096 entrées d'archive, 2 048 instances et deux millions de points/cellules. La profondeur des références d'instances est bornée. Ces plafonds sont des contrôles d'entrée ; ils ne garantissent pas qu'un fichier atteignant chaque plafond sera rapide ou tiendra dans la RAM disponible. L'API ou le projet portable peuvent imposer d'autres limites : prendre la plus restrictive du parcours utilisé.

Fichiers natifs avec convertisseur

Les formats natifs SOLIDWORKS, CATIA, Inventor, NX/Creo et plusieurs autres conteneurs sont reconnus comme **entrées conditionnelles**. Sans convertisseur local, ils sont refusés avec un diagnostic et une proposition d'export STEP. La liste exacte est dans `atelier/import_registry.py` et dans le catalogue de capacités.

Le mécanisme de conversion reçoit uniquement une commande définie par la configuration locale de l'installation. Un projet reçu, un `.cadpack` ou un fichier CAO ne peut pas définir la commande à exécuter. La liste d'arguments est lancée sans shell. Le convertisseur doit créer un STEP, IGES, BREP, STL ou MSH, dans les limites et le délai configurés ; la géométrie est encore validée par le noyau.

La passerelle fournie pour SOLIDWORKS demande Windows, PowerShell 64 bits STA, une installation desktop et des droits d'utilisation appropriés. **Elle n'a pas été essayée avec un SOLIDWORKS réel sur le Mac de l'audit.** Les neuf scénarios de protocole avec doubles ont été exécutés sous PowerShell sur Mac ; ce test ne constitue pas une certification de traduction ni un essai COM Windows. Voir IMPORTS_NATIFS.md pour la configuration `MAILLEUR_CAD_CONVERTERS`, les options, les erreurs, les assemblages supprimés/non résolus et les limites de version.

Transporter un assemblage : `.cadpack`

Un assemblage natif peut dépendre de dizaines de fichiers. Copier seulement son fichier principal dans une session ne les transporte pas. Le conteneur `.cadpack` rassemble une arborescence cohérente et un manifeste :

```
{"schema":"atelier.cadpack.v1","source":"assembly.sldasm"}
```

Préparer un dossier contenant ce fichier principal et toutes les références requises, puis créer le paquet :

```
.venv/bin/python -m atelier pack-cad --directory /chemin/assemblage \
  --source assembly.sldasm --output output/assemblage.cadpack
```

Les chemins du paquet sont relatifs. Les chemins absolus, remontées `..`, liens symboliques, archives chiffrées et paquets imbriqués sont refusés. Le manifeste ne répare pas les références internes de la CAO. Vérifier d'abord que l'assemblage se rouvre depuis le dossier préparé, par exemple après un Pack and Go SOLIDWORKS. Le convertisseur du fichier principal reste nécessaire sur la machine qui ouvre le `.cadpack`.

L'outil `pack-cad` sélectionne les fichiers dont les extensions CAO sont reconnues et signale le nombre d'autres fichiers omis. Il ne prétend pas embarquer des tables Excel, des textures ou toutes les ressources que le logiciel d'origine

pourrait référencer. Il ajoute les empreintes des fichiers sélectionnés au manifeste. Vérifier qu'aucune dépendance nécessaire à l'export n'a été omise ; à défaut, exporter un STEP autonome depuis le logiciel d'origine.

Un STEP autonome exporté par l'auteur reste une autre manière de transmettre la géométrie. Le paquet natif ne transporte pas son logiciel de création, sa licence, ni les droits de redistribution de ses pièces.

4. Générer, contrôler et exporter

Dans l'interface Qt, la sélection de source utilise le registre d'import et la liste d'export utilise `FORMATS`. Les formats conditionnels peuvent apparaître dans la boîte de sélection ; leur utilisation dépend encore du convertisseur annoncé par le catalogue. Contrôler les unités d'entrée, le mode, la réparation éventuelle et les zones avant le calcul.

Le rapport de génération indique si le maillage est accepté. Vérifier le minimum de qualité, les éléments médiocres et chaque région protégée. La marge d'une zone utilise une sélection conservatrice ; une zone nommée « contact » ne définit pas une loi de contact. La géométrie seule ne révèle pas les champs de contraintes d'un chargement inconnu.

Pour exporter une même pièce dans plusieurs formats, les identifiants suivants s'emploient avec `--formats`, séparés par des virgules :

Identifiant	Destination	Contenu et limites principales
`msh`	Gmsh MSH	Maillage natif d'échange du projet
`inp`	Abaqus	Nœuds, éléments et groupes selon les adaptateurs pris en charge
`med`, `unv`, `vtu`	Formats d'échange/visualisation	Selon les types et groupes documentés
`cast3m`	UNV et script d'import `.dgibi`	Import de maillage ; aucun calcul mécanique complet n'est créé
`ansys`	`.dat` historique, MESH200	Parcours de maillage existant ; distinct du nouvel export structurel
`nastran`	`.bdf`	Sous-ensemble structurel BDF, identifiants, ensembles et propriétés à compléter
`lsdyna`	`.key`	Éléments linéaires pris en charge, nœuds et ensembles ; matériaux/sections à définir
`ansys_structural`	`.apdl`	Types structurels explicitement associés aux éléments acceptés ; paramètres physiques à compléter

Les exports nouveaux ne visent pas tous les éléments imaginables. BDF prend notamment en charge les triangles/quadrangles, tétraèdres et hexaèdres d'ordres documentés, et le prisme linéaire. Le KEY se limite aux éléments linéaires pris en charge. L'APDL structurel distingue le plan, les coques et les volumes ; par exemple, le mode plan exporté doit être interprété selon son option de contrainte plane, pas comme une déformation plane ou une coque 3D par défaut. Une combinaison non validée est refusée. FORMATS.md précise les cartes, types, ordres, groupes et limites pour chaque écrivain.

Exemple :

```
.venv/bin/python -m atelier mesh examples/atelier_plate.json \
  -o output/plaque --formats msh,inp,nastran,lsdyna,ansys_structural
```

L'export convertit les unités depuis les mètres internes vers l'unité demandée, puis vérifie les artefacts et produit un manifeste. Ne pas définir une seconde conversion d'unité contradictoire dans le solveur. Les noms de groupes et les identifiants d'ensembles doivent être consultés dans les rapports, surtout quand un format distingue mal les ensembles de nœuds et d'éléments.

Une relecture par meshio ou un lecteur de sous-ensemble n'est pas une exécution dans Abaqus, ANSYS, LS-DYNA, Nastran ou Cast3M. Les essais de syntaxe et de connectivité doivent être complétés par l'ouverture et, si pertinent, un calcul minimal dans les versions cibles. Les preuves réellement exécutées et leurs dates restent dans les rapports de validation ; le présent guide ne les remplace pas.

5. Utiliser l'API locale

Lancer une instance locale, avec un seul travailleur pour commencer :

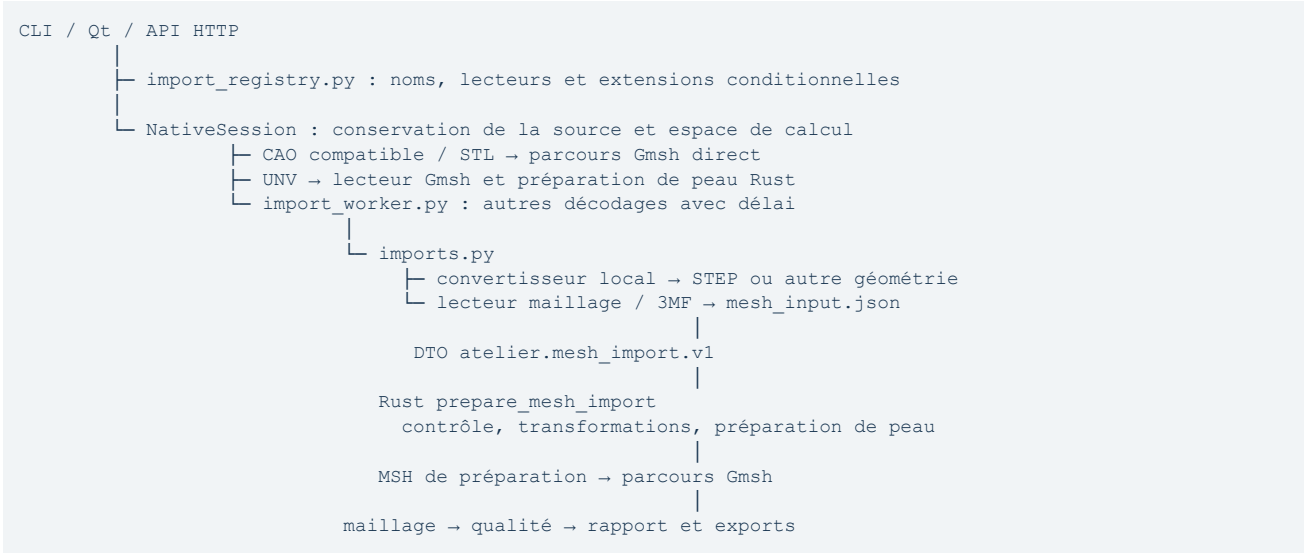
```
.venv/bin/python -m atelier serve --host 127.0.0.1 --port 8765 \
  --workspace output/api --workers 1 --timeout 300 --no-browser
```

Le catalogue est accessible par `GET /api/formats`. `GET /api/info` fournit aussi `import_formats`, objet de schéma `atelier.import_capabilities.v1`, avec une ligne par extension : `extension`, `kind`, `conditional`, `available`, et éventuellement `converter` et `reason`. Lire aussi `converter_configuration_error` : une erreur de configuration est différente d'un fichier CAO géométriquement invalide.

Le cycle général reste le suivant : lire les informations et le jeton de session, téléverser une source si nécessaire, créer un travail, suivre son état, puis récupérer rapport et export. Le fichier natif ou le `.cadpack` emprunte la même préparation que la CLI et Qt. Les limites du téléversement peuvent être inférieures aux limites internes du lecteur et l'encodage base64 augmente le volume transmis. Les erreurs de conversion restent des erreurs, pas des maillages réussis.

Les routes de travaux, sources, apprentissage, téléchargement et annulation sont détaillées dans API_UTILISATION.md. L'API HTTP n'est pas un serveur public multi-utilisateur prêt à héberger des CAO privées ; son modèle d'accès vise l'ordinateur local. Le jeton et les vérifications d'origine ne constituent pas une gestion des comptes, des droits et des espaces de travail séparés.

6. Architecture du parcours d'import



Élément	Rôle à préserver
`atelier/import_registry.py`	Catalogue léger utilisable par l'interface sans charger un traducteur CAO
`atelier/imports.py`	Validation des conteneurs, décodage des formats, choix du convertisseur local, sérialisation et provenance
`atelier/import_worker.py`	Exécution du décodage dans un processus distinct et écriture d'un diagnostic de préparation
`atelier/native.py` / `NativeSession`	Copie de la source, lancement des processus, délai, métadonnées, chaînage vers Rust et lecture des résultats
`atelier.mesh_import.v1`	Contrat JSON de points, cellules, objets, matrices et échelle ; aucun code exécutable reçu
`prepare_mesh_import` dans le noyau Rust	Validation numérique et géométrique du DTO, construction de la peau et du fichier de préparation
`atelier/formats.py` et `convert_worker.py`	Exports, conversions et vérifications propres au format de destination

Le pont vers Rust utilise des requêtes JSON sur disque et un processus `mailleur-core --request FICHIER`. Cette architecture n'est pas le serveur permanent JSON-lines de l'ancienne synthèse v2.5. Les fichiers de session permettent de distinguer erreur de lecture, convertisseur absent, source préparée invalide, maillage refusé et export impossible.

`import.json` décrit la préparation ; `converter.log` conserve la sortie du convertisseur lorsqu'il est utilisé. Le DTO temporaire s'appelle `mesh_input.json` et peut être supprimé après production du MSH canonique pour éviter une copie JSON volumineuse supplémentaire. La source originale et la source canonique restent conservées. Les métadonnées de session indiquent l'empreinte de l'intermédiaire ; le remaillage vérifie la géométrie, les unités, le mode et l'intégrité de cette préparation avant de la réutiliser.

La lecture d'un format et les opérations de représentation — par exemple ranger les coefficients d'une matrice 3MF dans le contrat attendu — ne créent pas un second moteur numérique Python. L'application effective des transformations, la qualité et la construction du maillage appartiennent à Rust et aux bibliothèques natives qu'il appelle.

7. Entraîner et partager un guide

Le parcours `learn` essaie des recettes sur la pièce importée comme sur une primitive compatible. Une stratégie retrouvée dans la mémoire est toujours exécutée et contrôlée de nouveau. Le classement neuronal ne peut pas rendre admissible un élément qui viole les règles de qualité.

Les guides de corpus et de mémoire sont des petits réseaux, pas des archives de toutes les pièces. Les tailles mesurées dans la livraison initiale sont détaillées dans TAILLES_ET_CAPACITES.md. Le nombre de configurations augmente surtout le corpus, les journaux et le coût de génération ; il ne multiplie pas mécaniquement le nombre de poids d'une architecture fixe.

Pour transmettre un guide compatible :

```
.venv/bin/python scripts/share_model.py --kind corpus \
  --model MON_CORPUS/models/current.json --output output/mon_guide.zip
```

Le paquet léger ne copie pas les CAO ni les journaux par défaut. Le guide de corpus s'installe dans l'espace de travail du destinataire selon le README du paquet ; celui-ci doit aussi installer le programme et ses dépendances. Un modèle valide mais désactivé reste désactivé.

Le guide interactif seul ne permet pas de reprendre une mémoire neuve : l'apprentissage cumulatif demande aussi les observations des groupes déjà appris. Transmettre une mémoire complète implique d'examiner ses sources et son historique. Une licence du code n'accorde pas automatiquement des droits sur les pièces d'entraînement d'autres auteurs.

8. Développer et optimiser avec des preuves

Commencer par un cas minimal partageable : fichier source, unité, mode, paramètres, configuration du convertisseur, version des bibliothèques, commande et résultat attendu. Pour un incident de natif, vérifier d'abord que la source et ses références s'ouvrent dans le logiciel d'origine. Un problème de traduction ne se corrige pas en assouplissant arbitrairement les seuils de qualité.

Pour ajouter un format, séparer les responsabilités : entrée dans le registre, décodage en DTO ou convertisseur explicitement configuré, validation numérique Rust, puis preuves sur les cellules et cas limites. Ne pas annoncer une famille de formats sur la seule réussite d'un fichier. Pour un export, documenter les types exacts, l'ordre des nœuds, les ensembles, les unités, les cartes physiques encore à définir et les refus attendus.

Les priorités d'optimisation concernent le poste effectivement coûteux :

1. **Mesurer par phase.** Distinguer lecture/décompression, copie de source, sérialisation DTO, conversion propriétaire, préparation Rust, Gmsh, contrôle de qualité, export et téléchargement. Comparer la médiane de plusieurs exécutions identiques en release ; séparer la première ouverture et les parcours réutilisés.
2. **Réduire les copies inutiles.** Mesurer le coût des tableaux Python, listes JSON et fichiers intermédiaires. Préférer une lecture par blocs pour les gros fichiers lorsque le format et les contrôles le permettent. Un fichier compact sur disque peut se dilater fortement pendant le décodage ; mesurer aussi le pic de mémoire.
3. **Réutiliser seulement ce qui reste valide.** Une clé de cache doit tenir compte du contenu de source, des dépendances d'assemblage, de l'unité, des transformations, de la version du lecteur/convertisseur et du contrat numérique. Le nom du fichier ou sa seule date ne suffisent pas. Modifier une recette ou une zone critique invalide les résultats qui en dépendent.
4. **Préserver les contrôles.** Ne pas supprimer les empreintes, ignorer une cellule, accepter un composant non résolu ou transférer aveuglément des anciens groupes pour gagner du temps. Une optimisation de copies ou d'allocations doit comparer les mêmes coordonnées, connectivités, unités et diagnostics.
5. **Limiter le parallélisme au besoin réel.** Les processus partagent RAM et disque. La passerelle SOLIDWORKS se sérialise et suppose une session sans documents ; multiplier les travailleurs ne la rend pas indépendante de ses contraintes de licence ou de session.

Les petits réseaux ne constituent pas la première cible si la traduction CAO ou la génération domine. Une modification de réseau exige des jeux réservés et un gain de sélection mesuré ; une accélération de son entraînement n'est pas une accélération automatique de tous les maillages.

Vérifications avant une contribution

Après installation, depuis un environnement où `python` et Cargo désignent les bons outils :

```
cargo test --locked --manifest-path rust_core/Cargo.toml
cargo build --locked --release --manifest-path rust_core/Cargo.toml
python -m atelier doctor
python -m unittest discover -s tests -v
```

Pour les tests Qt sans affichage, définir `QT_QPA_PLATFORM=offscreen` selon le shell utilisé. Exécuter d'abord les tests ciblés, puis les suites et parcours réellement concernés par le changement. Les tests de protocoles simulés de la passerelle PowerShell sont distincts d'un essai SOLIDWORKS réel.

La matrice GitHub Actions prévoit Linux, macOS et Windows avec Python 3.11 et 3.12. Sa présence ne prouve pas que ces six combinaisons ont été exécutées. Rapporter séparément tests locaux, CI effectivement terminée, relectures de fichiers et essais dans les logiciels propriétaires. Les anciens rapports Linux et Apple M3 restent datés de leurs campagnes ; ne pas les renommer pour les présenter comme des essais rc6.

9. Distribuer et publier

Le projet peut être préparé localement pour GitHub sans créer de dépôt distant. Le titulaire a choisi la **GNU GPL version 3 ou toute version ultérieure** (`GPL-3.0-or-later`) pour le code propre au projet ; le texte officiel figure dans LICENSE. Aucune identité d'auteur n'a été inventée. Consulter PUBLICATION.md avant de créer une release et THIRD_PARTY.md pour les composants tiers.

Une contribution utile indique le problème, le comportement corrigé, les formats et cas touchés, les vérifications, les mesures et les limites restantes. Joindre des exemples dont le partage est autorisé. Conserver les informations nécessaires pour reproduire un refus est aussi utile que joindre un maillage réussi.

Annexe A — Formats et exports solveurs

Un fichier de maillage contient des coordonnées, une connectivité et des groupes. Il ne remplace pas un modèle de calcul : matériau, formulation des éléments, épaisseur des coques, unités, chargements, contacts et conditions limites restent à définir dans le logiciel destinataire.

Formats disponibles

Export	Destination visée	Données et limites
Gmsh `.msh`	Gmsh et lecteurs Gmsh compatibles	Copie native avec groupes physiques et ordre des éléments conservés.
MED `.med`	Code_Aster, Salome-Meca, lecteurs MED, Cast3M selon version	Connectivité de dimensions mélangées. Groupes d'éléments sous forme de familles MED, y compris les groupes qui se recouvrent.
Abaqus/CalculiX `.inp`	Import de maillage Abaqus/CalculiX	Seuls les éléments de dimension maximale sont écrits comme éléments de calcul ; les groupes de frontière deviennent des ensembles de nœuds `NSET`.
VTK XML `.vtu`	ParaView	Maillage complet ; chaque groupe devient un masque `group:nom`. Visualisation et inspection, sans modèle de calcul.
I-DEAS Universal `.unv`	Salome, Cast3M et lecteurs UNV	Écriture et relecture par Gmsh ; vérifier la prise en charge de chaque type d'élément dans le logiciel cible.
ANSYS `.dat`	Mechanical APDL	Commandes de nœuds, éléments topologiques `MESH200`, composants d'éléments et de nœuds. Attribuer ensuite les types d'éléments de calcul dans ANSYS.
ANSYS structural `.apdl`	Mechanical APDL	Export distinct `ansys_structural`, avec types structuraux explicites selon le mode et les cellules. Matériaux, sections et analyse à compléter.
Nastran `.bdf`	Nastran et lecteurs Bulk Data compatibles	Export `nastran` : GRID*, cartes de coques/solides et ensembles SET1 ; propriétés PID et matériaux à définir.
LS-DYNA `.key`	LS-DYNA et lecteurs Keyword compatibles	Export `lsdyna` : nœuds, cellules linéaires et ensembles nommés ; définir les PART, SECTION/ELFORM et MAT.
Cast3M `.dgibi` + `.unv`	Cast3M	Script d'import `LIRE 'UNV'`, à lancer dans le dossier contenant le fichier UNV.

Les dépendances sont Gmsh 4.x, meshio 5.3.x, numpy et h5py pour MED. Un échec d'export apparaît dans le manifeste ; l'application ne présente pas cet export comme réussi. Le ZIP `exports.zip` ne contient que les fichiers effectivement produits, plus `export_manifest.json` et `groups.json`.

La cible demandée est l'échange avec les **versions 2020 et ultérieures**. Les écrivains utilisent des conventions documentées et des types explicites. Cela ne constitue pas une certification sur chaque logiciel, édition, version et physique : aucun banc des solveurs commerciaux 2020–2026 n'a été exécuté. Les formats absents de cette liste ne sont pas annoncés comme disponibles. Elmer, par exemple, n'a pas d'écrivain dédié dans cette révision ; un parcours via ElmerGrid demanderait une validation distincte.

Sélection et matrice de prise en charge

Depuis la racine, après installation :

```
.venv/bin/python -m atelier mesh examples/atelier_plate.json \
  -o output/echanges --formats msh,inp,nastran,lsdyna,ansys_structural
```

Les mêmes clés figurent dans `atelier.formats.FORMATS`, dans les choix de l'interface et dans `GET /api/info`. Vérifier le statut de chaque format et ses avertissements. Le format historique `ansys` conserve son rôle MESH200 et son extension `.dat` ; le nouveau structural utilise `.apdl`, pour que les deux sorties coexistent dans le paquet ZIP.

Cellule source	Nastran BDF	LS-DYNA Keyword	APDL structural
Triangle 3 nœuds	CTRIA3	ELEMENT_SHELL, N3=N4	PLANE182, K=L, mode plan seulement
Triangle 6 nœuds	CTRIA6	Refus	Refus : mapping non validé ici
Quadrilatère 4 nœuds	CQUAD4	ELEMENT_SHELL	PLANE182 en plan ; SHELL181 en surface
Quadrilatère 8 nœuds	CQUAD8	Refus	Refus : mapping non validé ici
Tétraèdre 4 nœuds	CTETRA	ELEMENT_SOLID, N4 répété	SOLID285, déplacement pur, KEYOPT(1)=1
Tétraèdre 10 nœuds	CTETRA, 10 nœuds	Refus	SOLID187
Hexaèdre 8 nœuds	CHEXA	ELEMENT_SOLID	SOLID185
Hexaèdre 20 nœuds	CHEXA, 20 nœuds	Refus	Refus : mapping non validé ici
Prisme 6 nœuds	CPENTA	ELEMENT_SOLID, N5=N6 et N7=N8	Refus
Autres, dont pyramides, quad9 et prisme15	Refus	Refus	Refus

Cette matrice décrit les écrivains, pas les capacités du générateur. Elle n'annonce pas que le moteur sait produire toutes ces formes sur une pièce quelconque. Certains contrats de connectivité utilisent des cellules de référence ; les tests d'intégration utilisent aussi des maillages Gmsh réels.

Choix de formulation dans INP

Pour un projet plan, l'export sélectionne `CPS3`, `CPS4`, `CPS6` ou `CPS8` : **contraintes planes**. Les déformations planes, l'axisymétrie ou un calcul thermique demandent de modifier la formulation dans le solveur. Une surface dans l'espace est exportée comme coque `S3`, `S4`, `S6` ou `S8R`. Les solides utilisent des éléments `C3D*`, notamment `C3D4` et `C3D10` pour les tétraèdres. Le fichier précise la formulation retenue dans son manifeste. Les pyramides linéaires utilisent `C3D5` pour Abaqus ; cet élément n'est pas présumé compatible avec CalculiX et un avertissement accompagne cet export.

Les éléments de frontière ne sont pas ajoutés comme poutres ou coques dans un calcul volumique. Leur groupe devient un ensemble de nœuds. Il n'existe pas de conversion automatique vers des faces de pression ou des surfaces de contact. Vérifier la version de CalculiX/Abaqus et les types autorisés pour son analyse.

ANSYS : topologie et physique séparées

Le format nommé `ansys` de meshio est un format de maillage Fluent. Il n'est pas utilisé ici pour prétendre produire un modèle Mechanical. Le script APDL écrit à la place `MESH200`, avec les options de forme documentées par ANSYS : triangles à 3/6 nœuds, quadrilatères à 4/8 nœuds, tétraèdres à 4/10 nœuds et hexaèdres à 8/20 nœuds. Ces éléments **n'ont pas de degrés de liberté** : le script importe la topologie, pas un modèle prêt à résoudre. Affecter les types de calcul adaptés dans Mechanical APDL avant d'effectuer une analyse.

Les groupes de frontière deviennent des composants nodaux préfixés `N_`. Les prismes et pyramides sont refusés par cet export plutôt que transformés silencieusement ou tronqués. Les autres formats peuvent les accepter.

APDL structural : parcours supplémentaire

`ansys_structural` choisit les types de la matrice. Les projets plans sont explicitement en **contraintes planes** et doivent être dans XY. Le triangle linéaire PLANE182 répète K et L, conformément à la référence ANSYS. Pour une coque dans l'espace, seul le quadrilatère SHELL181 est accepté : ANSYS réserve sa forme triangulaire dégénérée au remplissage, ce qui ne justifie pas d'en faire automatiquement tout un domaine triangulé.

SOLID285 est réglé en déplacement pur, pour élasticité linéaire ou très petites déformations plastiques. Ne pas reprendre cette formulation sans examen pour un matériau quasi incompressible ou une grande plasticité. Adapter la formulation à l'analyse. Les tétraèdres quadratiques conservent leurs dix nœuds dans SOLID187 et les hexaèdres linéaires utilisent SOLID185. Les autres formes sont refusées sans approximation cachée.

Le script ne crée ni matériau ni section fictive. Définir notamment les épaisseurs de coques avant résolution. Les composants nodaux et élémentaires partagent un espace de noms rendu unique ; le manifeste fournit `verification.component_name_map`. Le script est contrôlé structurellement, sans exécution ni relecture dans ANSYS.

Nastran : BDF, ordre et ensembles

`nastran` conserve les éléments de dimension maximale et transforme les groupes de frontière en ensembles nodaux, comme INP. Les cartes sont CTRIA3/6, CQUAD4/8, CTETRA, CHEXA et CPENTA. Les nœuds intermédiaires de l'hexaèdre à vingt nœuds sont permutés selon la convention Nastran ; une relecture meshio doit restituer exactement la connectivité de départ.

Les coordonnées sont écrites en champs **GRID* de 16 caractères**, avec point décimal et exposant implicite Nastran. La précision est finie : la relecture contrôle explicitement une erreur relative maximale de `5e-10` et une tolérance absolue de `1e-300`. Le manifeste conserve ces seuils et `maximum_coordinate_error`, erreur absolue réellement observée. La sortie n'est pas annoncée comme identique bit à bit. Cet écrivain ne change pas les unités : le pont natif a déjà appliqué l'unité d'export choisie.

Les groupes recouvrants deviennent des **SET1**. Une SET1 ne distingue pas intrinsèquement nœuds et éléments : le contexte du solveur détermine ce sens. Les commentaires `$ ATELIER_SET` et `verification.set_id_map` conservent l'identifiant, le nom et le type de chaque ensemble. Les identifiants solveur commencent à 1 ; les indices `groups.json` sont locaux aux blocs et commencent à 0. Les identifiants small-field sont limités à 99 999 999.

Un PID par type de cellule est exposé dans `verification.property_id_map`. Compléter ses PSHELL/PSOLID, matériaux et épaisseurs dans le solveur. Les appartenances aux groupes ne sont pas confondues avec les propriétés : un élément peut rester dans plusieurs SET1.

LS-DYNA : sous-ensemble Keyword linéaire

`lsdyna` écrit `*NODE`, `*ELEMENT_SHELL` ou `*ELEMENT_SOLID`. Il conserve les cellules de dimension maximale et transforme les groupes de frontière en ensembles nodaux. Les triangles utilisent N1,N2,N3,N3 ; les tétraèdres N1,N2,N3,N4,N4,N4,N4,N4 ; les prismes N1,N2,N3,N4,N5,N5,N6,N6, conformément au manuel Keyword. Les hexaèdres conservent leurs huit nœuds. Un type quadratique est refusé, sans supprimer ses nœuds intermédiaires.

Chaque type de cellule reçoit un PID, dans `verification.part_id_map`. L'export est un **fragment de maillage**, sans définition fictive de `*PART`, `*SECTION_*` ou `*MAT_*`. Ajouter ces cartes et choisir ELFORM dans le modèle de calcul. Le choix entre tétraèdre dégénéré et formulation tétraédrique, les énergies d'hourglass et la qualité mécanique relèvent de cette étape. Un domaine 2D sort comme topologie de coque ; cela ne sélectionne pas seul une formulation de mécanique plane LS-DYNA.

Les groupes nommés utilisent `*SET_NODE_LIST_TITLE`, `*SET_SHELL_LIST_TITLE` ou `*SET_SOLID_TITLE`. Les recouvrements restent conservés, sans création automatique de contacts, faces de pression ou PART. Un lecteur interne relit le sous-ensemble généré et contrôle coordonnées, connectivité et ensembles. Il n'est pas un lecteur LS-DYNA général et ne remplace pas l'ouverture dans LS-PrePost ou dans le solveur destinataire.

Groupes et ordre des éléments

Les groupes physiques Gmsh sont fusionnés par type d'élément lors des conversions. Cette opération conserve la connectivité et permet l'écriture MED, dont l'écrivain utilisé n'accepte qu'un bloc par type. Les appartenances qui se recouvrent sont conservées : un élément peut appartenir à `BOUNDARY` et à un groupe de face particulier.

Les noms sont convertis en ASCII simple quand le format l'impose et rendus uniques, sans ambiguïté de casse. Le dictionnaire `group_name_map` du manifeste conserve la correspondance avec les noms d'origine. `groups.json` conserve les indices par bloc de type fusionné, avec des indices commençant à zéro. Les exports solveur renumérotent les éléments ; ces indices ne sont donc pas des identifiants solveur.

Les éléments quadratiques restent quadratiques. Un type non accepté par un écrivain provoque une erreur explicite ; aucun export ne supprime les nœuds intermédiaires pour réussir artificiellement.

Ce qui est vérifié ici

- Copie MSH identique octet pour octet.
- Relecture MED et VTU, coordonnées, nombres, connectivité et ordre des nœuds.
- Conservation des groupes recouvrants MED et des masques VTU.
- INP : tests de relecture des triangles linéaires et tétraèdres quadratiques,

distinction éléments/ensembles nodaux, types explicitement choisis.

- UNV : écriture/relecture Gmsh et nombre d'éléments ; la sémantique des groupes

du lecteur final doit encore être contrôlée.

- APDL et Gibiane : contrôle structurel des scripts générés. ANSYS, Cast3M,

Abaqus, CalculiX et Code_Aster n'ont pas été exécutés dans cet environnement.

- BDF : neuf types, relecture meshio, numérotation CHEXA à vingt nœuds,

précision à plusieurs échelles et SET1 recouvrants/frontières contrôlés.

- Keyword : cinq formes linéaires, répétitions nodales documentées, groupes

recouvrants et refus des tétraèdres quadratiques sans écrasement de la sortie précédente ; relecture par le lecteur interne du sous-ensemble.

- APDL structural : SOLID285/187/185, PLANE182 et SHELL181 quadrangulaire,

composants sans collision et refus des formes non validées.

Les contrats supplémentaires sont dans `tests/test_solver_exports.py` et `tests/test_formats.py`. Aucun solveur commercial cible ni toute la série 2020+ n'ont été exécutés. Ces contrats ne valident pas un calcul mécanique.

La compatibilité avec tous les logiciels et toutes leurs versions ne peut pas être déduite de l'extension du fichier. Commencer par une petite pièce, vérifier les groupes et les unités, puis suivre les messages du logiciel cible.

Références officielles

- Gmsh : formats et API.
- meshio : formats et code source des écrivains.
- Code_Aster : maillage MED et Gmsh.
- CEA : Débuter avec Cast3M, import UNV.
- ANSYS : référence MESH200.
- ANSYS : SOLID285, SOLID187, SOLID185.
- ANSYS : PLANE182 et triangle K=L, SHELL181 et restriction des triangles.
- LS-DYNA : manuels officiels, Keyword Volume I R8, conventions ELEMENT_SOLID antérieures à 2020.

- PyDYNA, source ANSYS de SET_SOLID, SET_SHELL_LIST, SET_NODE_LIST.
- meshio : lecteur Nastran utilisé pour la relecture. Ces références définissent les conventions, sans prouver une exécution dans chaque version cible.

Annexe B — Imports natifs et passerelles

État du 7 novembre 2026. La passerelle fournie appelle les API officielles SOLIDWORKS pour produire un STEP que le noyau Mailleur peut ensuite importer. **Elle nécessite Windows et une installation SOLIDWORKS compatible ; elle n'est pas un lecteur `.sldprt` ou `.sldasm` intégré au moteur.** Son code n'a pas été exécuté sur une installation SOLIDWORKS réelle dans cet audit effectué sur Mac.

L'objectif de compatibilité concerne les logiciels de CAO à partir de leurs versions 2020. Ce périmètre constitue une cible de validation par éditeur, version et type de document, et non la garantie que tous leurs formats sont décodés. Un ancien logiciel de CAO n'est pas supposé ouvrir les fichiers natifs de toutes les versions ultérieures. Une lecture partielle en mode « future version » n'est pas une validation de traduction géométrique complète.

Choisir le chemin d'import

Source	Chemin vers Mailleur	Dépendance supplémentaire
STEP/STP, IGES/IGS, BREP	Import géométrique du moteur existant	Gmsh/OpenCASCADE livré par l'installation
Fichier de maillage ou de visualisation	Lecteur du format et reconstruction surfacique selon les restrictions documentées	Lecteur disponible et type d'élément pris en charge ; voir `FORMATS.md`
SOLIDWORKS `.sldprt`	Convertisseur configuré → STEP → import CAO	SOLIDWORKS Windows desktop et PowerShell 64 bits STA
SOLIDWORKS `.sldasm`	Même chaîne, avec toutes les références accessibles et la bonne configuration	Installation SOLIDWORKS compatible ; assemblage complet ou exclusions explicites
Autre format CAO natif	Convertisseur externe explicitement configuré → format géométrique intermédiaire	Logiciel de l'éditeur ou traducteur compatible, à valider
Fichier propriétaire sans convertisseur	Export STEP depuis son logiciel d'origine, puis import STEP	Aucune prise en charge native n'est annoncée en son absence

Un format reconnu dans une liste d'extensions peut donc être **conditionnel**. La présence de son nom ne prouve pas que le traducteur, la licence ou les références nécessaires sont disponibles. Le parcours doit signaler la conversion utilisée et refuser une sortie vide ou illisible. Une conversion d'un fichier de résultats éléments finis en surface n'importe pas automatiquement ses lois de matériaux, contacts ou chargements.

Ce que fournissent OpenCASCADE et FreeCAD

OCCT fournit des lecteurs/écrivains d'échange dont STEP, IGES et BREP ; ajouter un moteur OCCT ne suffit pas à démontrer la lecture universelle des formats natifs d'autres éditeurs. Les composants libres et les produits de traduction commerciale d'Open Cascade doivent être distingués : par exemple, les notes du produit séparé CAD Processor décrivent l'ajout de formats propriétaires. Cela ne prouve pas qu'ils soient présents dans la bibliothèque OCCT utilisée par Gmsh. Documentation OCCT, notes de CAD Processor.

FreeCAD annonce son format FCStd et de nombreux formats d'échange, notamment STEP, IGES et plusieurs maillages ; ses extensions peuvent ajouter d'autres formats. Une installation FreeCAD appropriée peut donc constituer une passerelle FCStd → STEP ou convertir les formats qu'elle lit réellement. Sa présence ne constitue pas à elle seule une preuve de prise en charge complète de `.sldprt`, CATPart, NX, Inventor ou Creo dans toutes leurs versions. Aucun convertisseur FreeCAD universel n'est fourni ou prétendu validé ici. Formats annoncés par FreeCAD.

Pour préserver une géométrie précise de solide, préférer le STEP produit par le logiciel d'origine plutôt qu'un STL de visualisation. Les deux représentations n'offrent pas les mêmes informations : une peau triangulée ne contient pas l'arbre paramétrique ni toutes les surfaces exactes d'origine.

Contrat du convertisseur configurable

Le fichier pointé par la variable `MAILLEUR_CAD_CONVERTERS` décrit les programmes locaux autorisés par la configuration de l'utilisateur. La commande est une **liste d'arguments**, et non une chaîne à exécuter dans un shell. `{input}` désigne la source de la session et `{output}` la destination intermédiaire à créer. Un code de sortie non nul, un dépassement du délai ou une sortie invalide fait échouer la conversion.

Exemple fourni : `scripts/converters/solidworks.example.json`. Adapter son chemin absolu au dossier réellement utilisé :

```
{
  "schema": "atelier.cad_converters.v1",
  "converters": [
    {
      "name": "SOLIDWORKS Windows COM vers STEP",
      "extensions": [".sldprt", ".sldasm"],
      "command": [
        "powershell.exe", "-NoProfile", "-STA", "-File",
        "C:/Mailleur/atelier_mailleur/scripts/converters/solidworks_to_step.ps1",
        "-InputPath", "{input}", "-OutputPath", "{output}"
      ],
      "output_extension": ".step",
      "timeout": 120
    }
  ]
}
```

Dans le terminal PowerShell utilisé pour lancer Mailleur :

```
$env:MAILLEUR_CAD_CONVERTERS = 'C:/Mailleur/atelier_mailleur/scripts/converters/solidworks.example.json'
```

Cette configuration ne télécharge ni SOLIDWORKS ni une licence. Elle ne doit contenir que des commandes locales dont l'utilisateur connaît la fonction. Ne pas ajouter une extension au convertisseur SOLIDWORKS pour prétendre lui faire lire un autre format natif : le script refuse les extensions hors `.sldprt` et `.sldasm`.

Exécuter la passerelle SOLIDWORKS seule

Interface exacte du script :

```
powershell.exe -NoProfile -STA -File .\scripts\converters\solidworks_to_step.ps1 `
  -InputPath 'C:\Pieces\piece.sldprt' -OutputPath 'C:\Echanges\piece.step'
```

Options supplémentaires :

- `-Configuration 'NomExact'` : demande une configuration précise. À défaut, SOLIDWORKS choisit celle ouverte selon le document ; le rapport donne son nom.
- `-AllowSuppressedComponents` : autorise explicitement l'exclusion des composants supprimés dans cette configuration. Sans ce paramètre, leur présence provoque un refus. Elle ne réactive aucun composant et ne permet pas d'ignorer des références manquantes ou des composants non résolus.

Le script écrit un rapport JSON compact sur la sortie standard en cas de succès, avec la révision de SOLIDWORKS, la configuration, les codes d'ouverture/export et le compte d'exclusions. Il retourne `0` uniquement après création du STEP. En cas d'échec, un diagnostic JSON est envoyé sur la sortie d'erreur et le code vaut `1`. Il refuse de remplacer un fichier de destination existant.

Utiliser PowerShell 64 bits en mode STA, dans une session Windows de bureau disposant de SOLIDWORKS desktop et de son API COM. La disponibilité de la classe COM `SldWorks.Application`, les droits de lancement et les fonctions d'export dépendent de l'installation. Respecter sa politique d'exécution PowerShell ; la passerelle ne la désactive pas. Le script n'est pas présenté comme validé pour un service Windows sans session interactive, SOLIDWORKS Connected, une licence Maker ou un poste distant automatisé.

Traitement effectivement demandé à SOLIDWORKS

1. Création de l'objet `SldWorks.Application`, sans attachement explicite à un document utilisateur. Si la session obtenue contient déjà des documents ou si la source est déjà ouverte, l'opération est refusée. Ne pas utiliser la même session SOLIDWORKS en parallèle pendant la conversion.
2. Ouverture en lecture seule par `OpenDoc6` avec les options Silent, ReadOnly et OverrideDefaultLoadLightweight, sans demander le chargement léger. Toute erreur ou tout avertissement d'ouverture est refusé ; le chemin retourné doit correspondre à la source. La disponibilité et la sémantique de ces options sont décrites par OpenDoc6 et swOpenDocOptions.
3. Activation du document sans reconstruction automatique. Une reconstruction nécessaire doit être réalisée et enregistrée dans le logiciel d'origine avant un nouvel essai. ActivateDoc3, options de reconstruction.
4. Pour un assemblage, résolution des composants légers puis contrôle des composants de la configuration active. Les configurations/composants SpeedPak, les références non chargées, les états incohérents et les composants restant légers sont refusés. Les composants supprimés sont refusés sauf exclusion explicitement autorisée. Il n'y a aucune activation automatique de composants ou substitution de fichiers. Résolution des composants, états des composants, GetComponents, SpeedPak.
5. Effacement de la sélection avec `ClearSelection2`, puis export par `IModelDocExtension.SaveAs` vers un fichier STEP temporaire. Cette API stable antérieure à 2020 reste documentée mais est marquée obsolète au profit de méthodes plus récentes ; elle a été choisie pour conserver un contrat ancien et simple. Les faces sélectionnées pourraient sinon restreindre l'export. Le script exige un retour vrai, aucun code d'erreur et aucun avertissement d'export, puis contrôle l'existence et la signature du fichier avant de le publier. SaveAs et conditions d'export STEP, documentation récente de SaveAs.

Les API utilisées sont documentées avant ou dans la génération 2020 ; leur disponibilité documentaire ne remplace pas un essai avec chaque version installée et chaque type de fichier.

Avant l'ouverture, le script positionne le répertoire courant SOLIDWORKS sur celui de la source et restaure sa valeur initiale lors du nettoyage. Les dossiers de recherche personnalisés restent prioritaires selon SOLIDWORKS ; vérifier que le logiciel trouve les bonnes références, en particulier si plusieurs versions portent le même nom. Répertoire de recherche utilisé par OpenDoc6.

Le script n'appelle ni sauvegarde native de la source, ni fermeture de tous les documents, ni arrêt de SOLIDWORKS. Il ferme seulement le document qu'il a ouvert après contrôle de son chemin. Une session SOLIDWORKS peut rester démarrée. Si le délai du processus hôte expire brutalement ou si SOLIDWORKS ouvre une boîte de dialogue bloquante, son nettoyage normal peut ne pas s'exécuter ; examiner alors la session avant de relancer. Le mutex du script empêche deux conversions de cette passerelle de se lancer simultanément, sans verrouiller l'interface contre un utilisateur.

Les préférences d'export STEP déjà configurées dans SOLIDWORKS ne sont pas modifiées. Il faut contrôler le STEP obtenu, ses unités, ses solides, ses instances et son orientation dans Mailleur, puis la qualité du maillage. L'arbre de construction paramétrique, les contraintes d'assemblage, les lois de contact et les données d'un calcul mécanique ne sont pas importés par cette traduction géométrique.

Références d'assemblage et `.cadpack`

Un fichier `.sldasm` peut référencer des pièces dans d'autres fichiers. Transmettre uniquement ce fichier au serveur ou le copier dans une session isolée ne transporte pas les pièces référencées. La même difficulté existe pour certaines pièces dérivées et leurs références externes. Un STEP complet et autonome exporté en amont reste le parcours le plus simple à partager.

Le conteneur Mailleur `.cadpack` est un ZIP de transport contenant `cadpack.json`, la source et ses dépendances dans une arborescence relative. Exemple de manifeste :

```
{"schema":"atelier.cadpack.v1","source":"assembly.sldasm"}
```

Exemple d'arborescence :

```
assembly.cadpack
├── cadpack.json
├── assembly.sldasm
└── parts/
    ├── body.sldprt
    └── support.sldprt
```

Préserver les références à ces fichiers ; ajouter simplement des pièces portant le bon nom dans le ZIP ne répare pas des liens absolus ou des configurations absentes. La fonction officielle Pack and Go de SOLIDWORKS permet de rassembler les fichiers liés. Préparer un dossier cohérent, vérifier qu'il s'ouvre correctement dans SOLIDWORKS, puis ajouter le manifeste et créer le conteneur Mailleur. Un ZIP Pack and Go sans ce manifeste ne devient pas automatiquement un `.cadpack` valide.

La validation du conteneur Mailleur porte sur les chemins relatifs, les limites et les fichiers présents. Elle ne prouve pas que toutes les références internes de l'éditeur sont résolues : le convertisseur doit encore refuser les avertissements et vérifier le document ouvert. Le `.cadpack` ne contient pas le logiciel CAO ni sa licence.

Validation à réaliser sous Windows

Le fichier `scripts/converters/test_solidworks_bridge.ps1` fournit neuf scénarios de protocole avec doubles PowerShell : succès, session occupée, exclusion refusée/autorisée, avertissement d'ouverture, erreur de sauvegarde, propriété du document, signature de sortie et refus d'écrasement. Ces neuf scénarios ont été exécutés avec succès le 7 novembre 2026 sous **PowerShell 7.6.6 sur Mac arm64**, avec une distribution portable officielle temporaire. Ils ne lancent pas SOLIDWORKS et ne valident ni COM Windows, ni une conversion industrielle réelle. Le journal est conservé dans `verification/rc6_20261107/solidworks_mock_tests.txt`.

```
powershell.exe -NoProfile -File .\scripts\converters\test_solidworks_bridge.ps1
```

Avant d'annoncer un support SOLIDWORKS opérationnel, compléter ces tests simulés par un essai réel sur au moins une pièce, un assemblage à références externes et un assemblage à composants supprimés. Vérifier l'absence de modification de la source et d'effet sur les documents préexistants, le nombre de corps/solides, les unités et la géométrie importée. Faire ces essais pour les versions de SOLIDWORKS effectivement visées ; la réussite des doubles et la lecture de la documentation ne suffisent pas à les déclarer réussis.

Licence et distribution

Le projet distribue un script d'automatisation, sans binaire, SDK ni licence SOLIDWORKS. Le poste qui le lance doit disposer d'une installation et de droits permettant cet usage et l'export des documents concernés. Les droits sur les géométries et leurs dépendances restent ceux de leurs auteurs. Redistribuer Mailleur n'autorise pas à redistribuer les logiciels propriétaires nécessaires aux convertisseurs. Les règles et droits d'un traducteur externe différent doivent être documentés séparément.

Annexe C — Publication et contribution

Ce document décrit une préparation locale. Il ne crée pas de dépôt distant et ne publie pas de fichiers. Le titulaire a choisi la **GNU GPL version 3 ou toute version ultérieure** pour le code propre au projet.

État et périmètre à annoncer

La révision rc6 ajoute des lecteurs et une passerelle locale de conversion, ainsi que des exports de sous-ensembles structurels. Le catalogue `python -m atelier formats --json` expose les capacités de la version installée. Présenter séparément :

- les formats intégrés et leurs éléments réellement acceptés ;
- les formats conditionnels, qui demandent un convertisseur configuré et éventuellement un logiciel propriétaire ;
- les vérifications de fichiers et tests effectivement exécutés ;
- les essais encore nécessaires dans les versions des logiciels cibles.

La cible 2020+ n'est pas une certification universelle. Un lecteur Nastran de maillage n'est pas un parseur complet de tout jeu de commandes Nastran ; un `.key` ou un `.apdl` de maillage n'est pas un calcul mécanique complet. La passerelle SOLIDWORKS est conditionnelle : ses neuf scénarios de protocole avec doubles ont été exécutés sous PowerShell sur Mac, mais aucune conversion avec un SOLIDWORKS réel n'a été validée. Les résultats historiques Linux et Apple M3 du 4 novembre 2026 doivent conserver leur date et leur révision d'origine.

Licence retenue : GPL-3.0-or-later

Le code propre au projet est distribué sous la **GNU GPL version 3 ou toute version ultérieure**, selon le choix explicite du titulaire. Le texte officiel figure dans LICENSE et l'identifiant de licence est `GPL-3.0-or-later`. Conserver cette information et le texte de licence dans les distributions. Aucun nom d'auteur, année de création ou détenteur de copyright n'a été inventé pour remplir une notice.

Le choix de la licence du code personnel ne remplace pas les obligations applicables à Gmsh, OpenCASCADE, Qt/PySide et aux autres bibliothèques. Examiner THIRD_PARTY.md et les notices des versions effectivement distribuées. Un paquet source qui installe les dépendances et une application qui embarque leurs binaires n'ont pas le même contenu à inventorier. Les traducteurs commerciaux et SOLIDWORKS ne sont pas distribués par la passerelle fournie.

Traiter aussi les droits sur les données : exemples CAO, fichiers natifs, pièces incluses dans les assemblages, jeux d'entraînement et poids. Une licence du programme n'autorise pas à publier une pièce confidentielle ou le corpus d'un tiers. Les `.cadpack` et dossiers d'expérience peuvent contenir bien plus que le seul nom du projet.

Contenu du dépôt

Inclure les éléments nécessaires pour comprendre, installer, tester et reproduire les exemples publics :

À conserver	Pourquoi
`README.md`, documentation et notices	Installation, architecture, formats, API, limites et contribution
`atelier/`, `mailleur/`, `rust_core/src/`	Sources de l'interface, du transport, des conversions et du noyau
`pyproject.toml`, `requirements.txt`, Cargo manifest et verrou	Dépendances et versions reproductibles
Lanceurs et `scripts/`	Installation, diagnostics, mesures et outils de distribution
`tests/` et leurs fixtures publiques	Vérification des contrats et cas limites
`examples/` partageables	Première génération et usage reproductible

À conserver	Pourquoi
`.github/workflows/`	Définition des vérifications automatisées
Preuves sélectionnées de `verification/`	Origine des résultats annoncés et fixtures utilisées par certains tests

Ne pas supprimer intégralement `verification/` sans examiner les dépendances des tests : des exemples de modèles et fichiers de validation y servent de fixtures. On peut alléger les historiques de campagnes dans une release, à condition de conserver les preuves citées et les fichiers nécessaires aux tests annoncés. Expliquer le contenu exact de l'archive.

Exclure les environnements `.venv`/`.runtime`/`.bootstrap`, la chaîne locale `.toolchain`, les caches Cargo et Python, `target/`, les travaux de `output/`, les corpus de production et les CAO privées. Le `.gitignore` est une aide ; il ne peut pas déduire quels fichiers placés sous un autre nom sont confidentiels. Examiner également les PDF, captures et rapports générés pour leurs noms de fichiers et chemins locaux.

L'outil `python scripts/inventory_sizes.py` sépare sources, preuves et postes créés par l'installation. Un corpus ou environnement de plusieurs gigaoctets n'a pas à entrer dans l'historique Git courant pour permettre l'installation du programme.

Vérifier une extraction propre

Installer et tester depuis une copie ou une extraction neuve avec les dépendances documentées. Le fait que le dossier de développement fonctionne ne prouve pas que tous ses fichiers nécessaires sont présents dans le paquet.

Exécuter les vérifications correspondant au contenu annoncé : compilation Rust avec verrou Cargo, tests, diagnostic, génération d'un exemple, import étendu, export de sous-ensemble et relecture. Pour un format conditionnel, garder un résultat « convertisseur absent » clair et tester séparément le parcours réel lorsque son logiciel est disponible.

Enregistrer version du projet, système/architecture, versions de Python, Rust, Gmsh et bibliothèques, commandes, nombres de tests, éventuels tests ignorés et résultat. Pour les performances, conserver les mesures brutes et séparer préparation, compilation, traduction, maillage, qualité et export. Ne pas recopier les chiffres d'une autre machine dans un rapport local.

La CI définie dans tests.yml est une matrice Linux/macOS/Windows avec Python 3.11 et 3.12. Une coche de workflow et un compte de tests doivent provenir d'une exécution réelle de cette version. Les plateformes sans logiciel propriétaire ne peuvent certifier sa passerelle par un simple test simulé.

Préparer un premier commit local

Si le dossier n'est pas encore suivi par Git, initialiser un dépôt local après examen des fichiers. Ces commandes sont des instructions à adapter au dossier de travail ; elles ne publient rien :

```
git init
git status --short
git diff --stat
```

Ajouter ensuite explicitement les fichiers publics retenus, puis examiner `git diff --cached --stat` et `git diff --cached` avant le commit. Vérifier les nouvelles sources, les binaires éventuels et les données de validation, plutôt que d'ajouter indistinctement tout le dossier de développement. Si des fichiers ont déjà été suivis, les nouvelles règles d'exclusion ne les retirent pas de l'historique.

Créer le dépôt distant, définir sa visibilité et pousser le commit constituent une étape séparée, à effectuer après examen du contenu et des droits sur les données. La préparation de cette révision n'a pas créé de dépôt GitHub ni transmis de CAO à un service de publication.

Préparer une release

Le script `scripts/package_native_release.py` prépare une archive source et son manifeste d'intégrité selon les règles du projet. Vérifier sa sortie, les rapports qu'il embarque et le binaire propre à la plateforme. Un binaire macOS arm64 ne fonctionne pas directement sous Windows ou Linux ; les sources peuvent être compilées sur le système cible.

Une release devrait contenir : le numéro de révision exact, ce qui a changé, la liste des formats avec leur niveau de validation, les prérequis et commandes de démarrage, les résultats effectivement obtenus, les limites, les notices et les empreintes des artefacts. Une archive d'intégrité permet de détecter une altération ; elle ne prouve pas à elle seule l'identité d'un auteur.

Les PDF et documents Word se reconstruisent séparément du moteur, avec les dépendances facultatives de documentation :

```
python -m pip install -r requirements-docs.txt
python scripts/build_guide.py --public-release --check-sources
python scripts/build_guide.py --public-release
```

Le guide rc6 assemble cinq sources, dont `VALIDATION_RC6.md` obligatoire, et calcule le nombre de chapitres/annexes. Les sorties `Mailleur_Guide_public_rc6.pdf` et `.docx`, le relevé de contrôle textuel et le manifeste d'empreintes sont créés dans `docs/livrables/`, avec des noms distincts des anciens guides rc5. Le générateur utilise les polices installées ou le repli Vera/Courier fourni par les outils, sans télécharger de polices. `--fonts DOSSIER` permet de choisir un dossier compatible. Pour l'ancienne collection, `--historical-rc5 --source-directory DOSSIER_ARCHIVE` demande des sources historiques conservées ; l'option ne restaure pas d'anciennes versions de fichiers.

Distribuer un guide appris avec `scripts/share_model.py` si le destinataire n'a pas besoin du corpus. Ce paquet conserve les métadonnées et l'état d'activation, sans importer automatiquement la mémoire ni ses sources. Pour une mémoire interactive, les poids seuls ne suffisent pas à reprendre un historique cumulatif sur un espace neuf. Pour un corpus, indiquer la provenance et les jeux de validation/test, sans annoncer qu'un million de cas a été exécuté parce que le plan le prévoit.

Recevoir et examiner les contributions

Demander un exemple partageable, la commande, les unités, les versions, le rapport obtenu et le résultat attendu. Pour une conversion propriétaire, demander la version du logiciel et l'état de ses références, sans exiger une pièce dont le partage est interdit. Favoriser une reproduction synthétique lorsqu'une géométrie est confidentielle.

Une proposition doit expliquer le comportement après correction et ses vérifications. Comparer les connectivités, groupes et unités, en plus du rendu visuel. Une optimisation ne doit pas effacer un avertissement, accepter un élément refusé ou prétendre transférer des données physiques absentes. Les ajouts de formats, de traducteurs et de commandes doivent être explicites dans le catalogue et la documentation.

Voir UTILISATION_ET_DEVELOPPEMENT.md pour les responsabilités du code et la méthode de mesure, et CONTRIBUER.md pour les règles de contribution existantes.

Annexe D — Validation de la rc6

Résultats effectivement obtenus

Cette révision a été compilée et testée sur **MacBook Pro Apple M3, 16 Go, macOS 27.0, arm64**. Python 3.12.16, Rust 1.99.1 et Gmsh 4.15.2 sont utilisés. Les dépendances Python de calcul restent celles de la préparation M3 ; aucun SDK CAO propriétaire n'est installé ni redistribué dans cette livraison.

Vérification	Résultat observé
Suite Python complète	**195 tests réussis en 25,242 s**, aucun échec ni test ignoré
Suite Rust complète, profil release	**121 tests réussis en 4,89 s**, aucun échec ni test ignoré
Compilation Rust	Release native Apple M3, Cargo.lock utilisé, version `1.0.0-rc.6`
Diagnostic et dépendances	Noyau et Gmsh trouvés ; `pip check` sans dépendance cassée
Passerelle SOLIDWORKS, doubles de test	**9 scénarios réussis** avec PowerShell 7.6.6 portable sur Mac
Conversion avec SOLIDWORKS installé	**Non exécutée** ; Windows et SOLIDWORKS requis
Calculs dans Abaqus, Cast3M, ANSYS, Nastran ou LS-DYNA	**Non exécutés** ; aucun résultat solveur n'est revendiqué

Les durées des suites n'incluent pas la compilation. Elles ne mesurent pas un calcul industriel, une traduction CAO ou la qualité d'une solution mécanique. La matrice CI proposée pour Linux/macOS/Windows n'a pas été exécutée à distance pendant cette préparation locale.

Les journaux complets et les versions sont dans `verification/rc6_20261107/`. Le fichier `summary.json` en donne le bilan structuré. Les rapports du 4 novembre restent dans leurs dossiers d'origine ; ils ne sont pas utilisés comme preuve de ces nouveaux formats.

Imports de fichiers réellement exercés

Les nouveaux tests utilisent de petites géométries publiques construites dans un dossier temporaire. Ils demandent un véritable nouveau maillage au noyau Rust/Gmsh, vérifient l'admissibilité et contrôlent que l'original est inchangé.

Entrée	Parcours contrôlé
OBJ, PLY, OFF	Lecture d'une peau tétraédrique et génération volumique ; quadrilatère OBJ remaillé en 2D
VTU, VTK, MSH	Lecture de cellules volumiques, extraction de leur peau et nouveau maillage
Abaqus INP, MED, Nastran BDF	Lecture de maillages tétraédriques, reconstruction et génération
Medit MESH, Netgen VOL	Lecture de cellules tétraédriques, reconstruction et génération
UNV volumique	Lecture Gmsh native, extraction de peau et nouveau maillage admissible
3MF Core	Objets, composants internes, unités et chaîne de transformations ; vérification des positions dans le résultat natif
Entrée quadratique VTU	Reconstruction par les sommets principaux, approximation explicitement signalée
STEP et STL	Régressions existantes, unités 3D, transformations et vérifications du parcours natif

L'essai API transmet aussi un OBJ par upload HTTP, lance sa reconstruction, attend le travail réel et télécharge un BDF contenant des tétraèdres. Le catalogue des formats est testé sur `/api/formats` et `/api/info`.

Les cas de refus vérifiés couvrent les références XML ou fichiers externes interdits, les graphes 3MF cycliques ou vides, les extensions requises non prises en charge, les liens HDF5 et les commandes INCLUDE externes. Les conteneurs CADPACK sont vérifiés pour les chemins, liens, collisions de casse, noms non portables, fichiers absents et empreintes altérées.

Le noyau teste connectivités, coordonnées non finies, faces internes, cellules dégénérées, orientation et topologies non manifold. Les objets 3MF dont les boîtes se touchent ou se chevauchent sont refusés conservativement. Ce contrôle peut rejeter des formes pourtant séparées ; il n'effectue pas d'union booléenne.

Exports et vérifications

Les exports historiques restent dans la suite. Les trois nouveaux profils sont testés séparément, avec refus des types qu'ils ne représentent pas.

Profil ajouté	Contrôles réalisés
`nastran` / BDF	Cartes documentées, relecture meshio, numérotation des nœuds quadratiques, précision des coordonnées, ensembles et correspondances de noms
`lsdyna` / KEY	Nœuds, cellules linéaires et répétitions de nœuds selon forme ; relecture du sous-ensemble écrit et ensembles ; refus des quadratiques sans réduction d'ordre
`ansys_structural` / APDL	Types SOLID285, SOLID187, SOLID185, PLANE182 et SHELL181 selon cellule/mode ; connectivité et composants ; refus des formes non prises en charge

La relecture d'un sous-ensemble KEY par le lecteur de contrôle fourni n'est pas une validation par LS-DYNA. Un script APDL contrôlé structurellement n'est pas un calcul exécuté dans ANSYS. Une relecture BDF préserve des données de maillage, sans prouver qu'un jeu de cartes complet possède les propriétés, matériaux et conditions nécessaires à une analyse Nastran.

Le bundle de tous les profils est testé sur un volume linéaire. Sur un type non accepté, un format peut apparaître en erreur dans le manifeste du bundle, tandis que les autres exports restent disponibles. Par exemple, LS-DYNA ne reçoit pas une réduction silencieuse des tétraèdres quadratiques.

Annulation, délai et intégrité des sessions

Les essais utilisent aussi de vrais processus de test qui ignorent SIGTERM. Ils contrôlent l'arrêt des descendants après annulation et après un timeout interne du convertisseur, même si le processus principal s'est déjà terminé. L'API nettoie le groupe qu'elle possède à la fin de chaque travail.

Une requête concurrente ne peut pas réinitialiser l'annulation active. L'opération partage une échéance entre lecture, normalisation et maillage. Une source dépassant 200 Mio est refusée avant copie ; l'API conserve sa limite d'upload distincte de 50 Mio. Un fichier préparé altéré n'est pas réutilisé pour un remaillage local. Ces limites ne constituent pas une borne absolue de RAM pour chaque bibliothèque et chaque fichier compressé.

Sens de la cible 2020+

Le projet vise des échanges avec les versions 2020 et ultérieures à travers des formats et types documentés. **Il n'est pas certifié pour tous les logiciels, toutes les variantes et toutes les versions depuis 2020.** Aucun test local de fichier ne peut remplacer cette matrice d'essais externes.

Les formats natifs restent conditionnés à une passerelle adaptée au fichier. Les tests SOLIDWORKS simulent les API ; ils ne prouvent pas que SOLIDWORKS 2020 lit un assemblage créé dans une version plus récente. Une installation réelle doit encore vérifier la résolution des composants et la géométrie STEP obtenue.

Pour contribuer à cette validation, partir d'une petite pièce partageable, noter les versions exactes, contrôler nombres d'éléments, connectivités, groupes, unités et orientation dans le logiciel cible, puis effectuer un petit calcul pertinent.

Conserver les refus et différences avec les réussites. Un futur résultat de logiciel cible doit compléter ces preuves, pas être déduit de son extension de fichier.

Commandes de reproduction

Après installation, utiliser le Python de `.venv` et une chaîne Cargo accessible :

```
cargo test --locked --release --manifest-path rust_core/Cargo.toml
python -m unittest discover -s tests -v
python -m atelier doctor
python -m atelier formats --json
```

Pour les contrôles de protocole sans SOLIDWORKS :

```
pwsh -NoProfile -NonInteractive -File scripts/converters/test_solidworks_bridge.ps1
```

Sous Windows PowerShell, remplacer `pwsh` par `powershell.exe`. Cette commande de test ne demande pas COM ; le script de conversion réel demande une session Windows STA et l'installation SOLIDWORKS décrites dans `IMPORTS_NATIFS.md`.